SWIFTDROP · DESIGN DOCUMENTATION · KAVINU SAPUTHANTHRI

Merchant Onboarding and API

Design of Kavinu Saputhanthri's part: online stores register, an admin approves them, and they create delivery orders from their own systems with OAuth 2.0 client credentials.

Services merchant-service (Spring Boot 4) · merchant orders in parcel-service **Data** PostgreSQL merchants_db · parcels_db · Kafka **Issue** #20

1 Use cases

2 Data model

3 Architecture

4 Integration design

5 Key flows

6 API design note

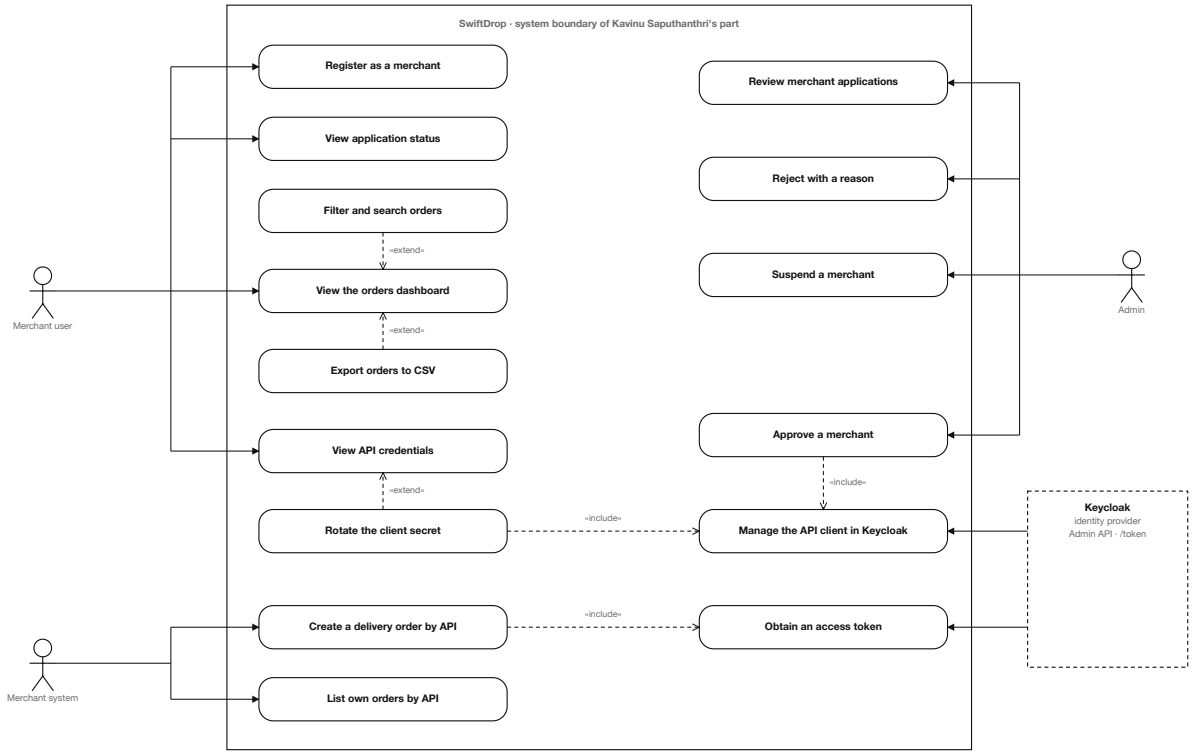
7 Design patterns and principles

Diagram sources: `doc/diagrams/m7/` in the SwiftDrop repository, with matching PNG exports. This document is the design for issue #20; it describes the planned behaviour. Example values are illustrative.

1 Use cases

Kavinu Saputhanthri owns merchant onboarding and the merchant API. An online store **registers**, an admin **approves** it, it receives **OAuth 2.0 client credentials**, creates delivery orders from its own system, and sees every order in a dashboard.

FIGURE 1 **Use cases: merchant onboarding and the merchant API**



A merchant user works through the portal, a merchant system through the API, and an admin decides every application. Keycloak is the external system behind provisioning, rotation and tokens. «include» points to the use case that is always included; «extend» points to the use case it extends.

2 Data model

Each service owns its data. `merchants_db` holds the application and a record of the API client, while the client secret stays in Keycloak. `parcels_db` gains `IDEMPOTENCY_RECORD`, plus `MERCHANT_STATUS_CACHE`, built from lifecycle events so an order never needs a call to merchant-service.

FIGURE 2 **Data model: merchants, credentials and idempotency records**



Solid crow's-foot lines are foreign keys inside one database. Dashed lines cross a service boundary by id or by event, never by foreign key. No client secret is stored in either database. PARCEL shows only the columns the merchant API uses; M3 owns the full table.

3 Architecture

Proposal: microservices behind an API gateway, with the Next.js app acting as backend-for-frontend (BFF). Each service owns its data; services integrate through Kafka events or through the gateway.

Why this style for merchant onboarding

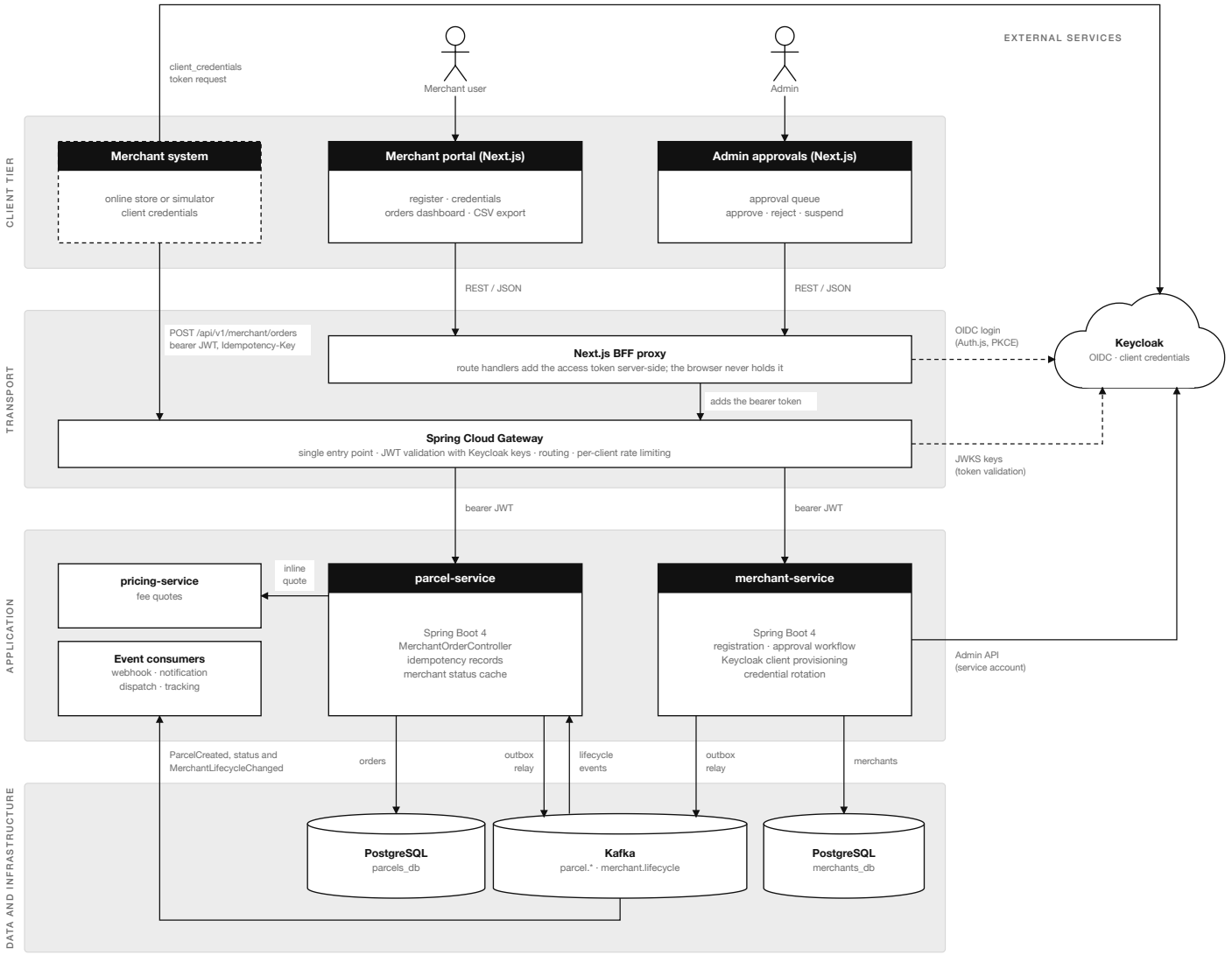
- **Separate failure domains:** if merchant-service or Keycloak's Admin API is down, approved merchants keep placing orders, because parcel-service reads its own status cache.
- **Independent scaling:** order traffic from online stores can burst far above the occasional onboarding request.
- **Clear ownership:** merchant-service owns the merchant's lifecycle; parcel-service owns the parcel. A merchant order is a parcel, so the order API belongs with the parcel aggregate.
- **Security by construction:** only merchant-service holds the Keycloak admin account, and machine clients receive no more than the `MERCHANT_API` role.

Tiers

Client	Merchant portal, admin approvals (Next.js), and the merchant's own system
Transport	Next.js BFF and Spring Cloud Gateway: one entry point, JWT validation
Application	merchant-service and parcel-service (api → application → domain → infrastructure)
Data	PostgreSQL `merchants_db` and `parcels_db`, Kafka topics

Trade-off accepted: the status cache is eventually consistent, so a suspension reaches the order API moments after it is applied.

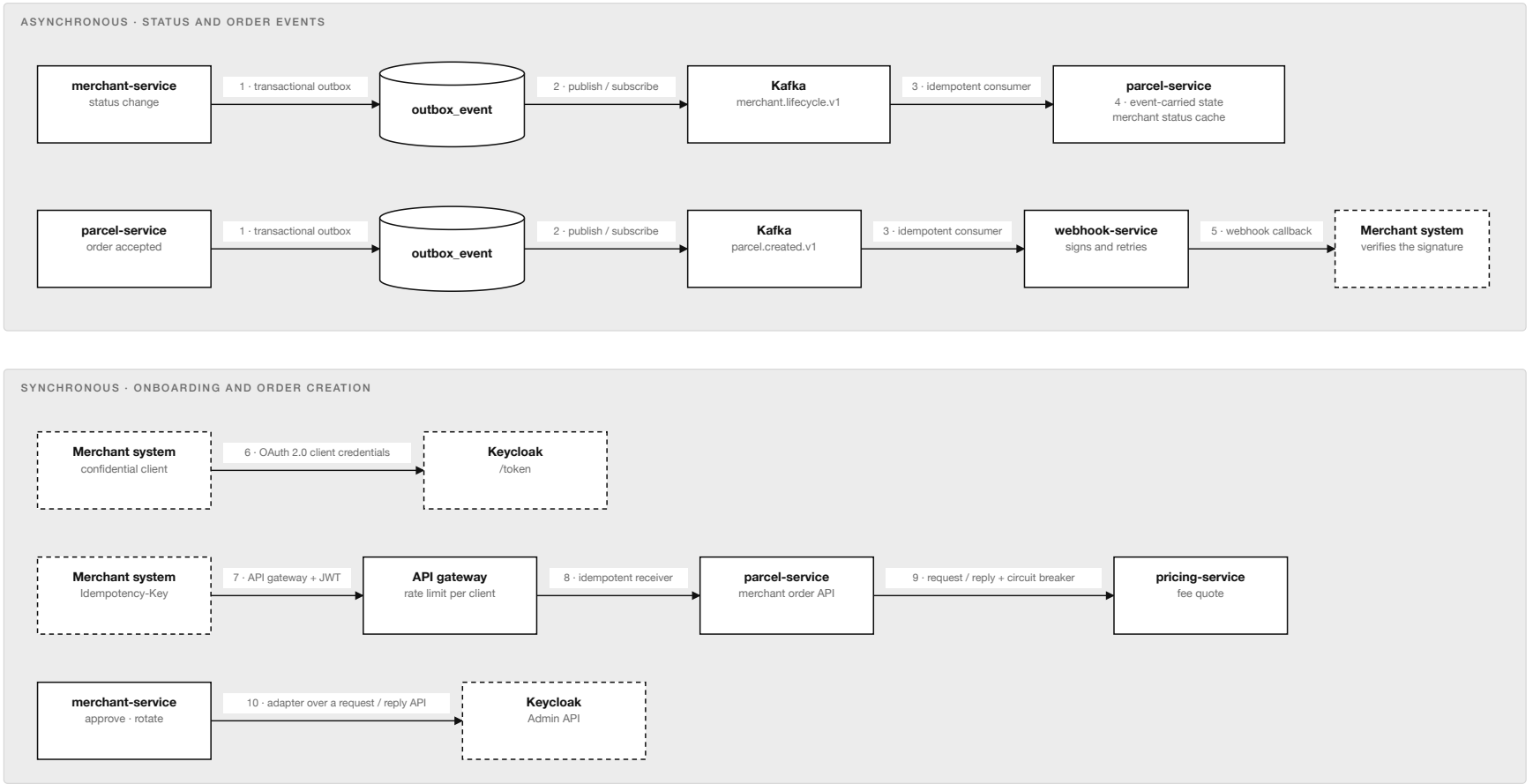
FIGURE 3 **Architecture: layered microservices with a BFF**



Four tiers. People reach the services through the BFF; the merchant's own system calls the gateway directly with a client-credentials token. merchant-service is the only service that calls the Keycloak Admin API.

4 Integration design

FIGURE 4 **Integration design: the patterns behind onboarding and orders**



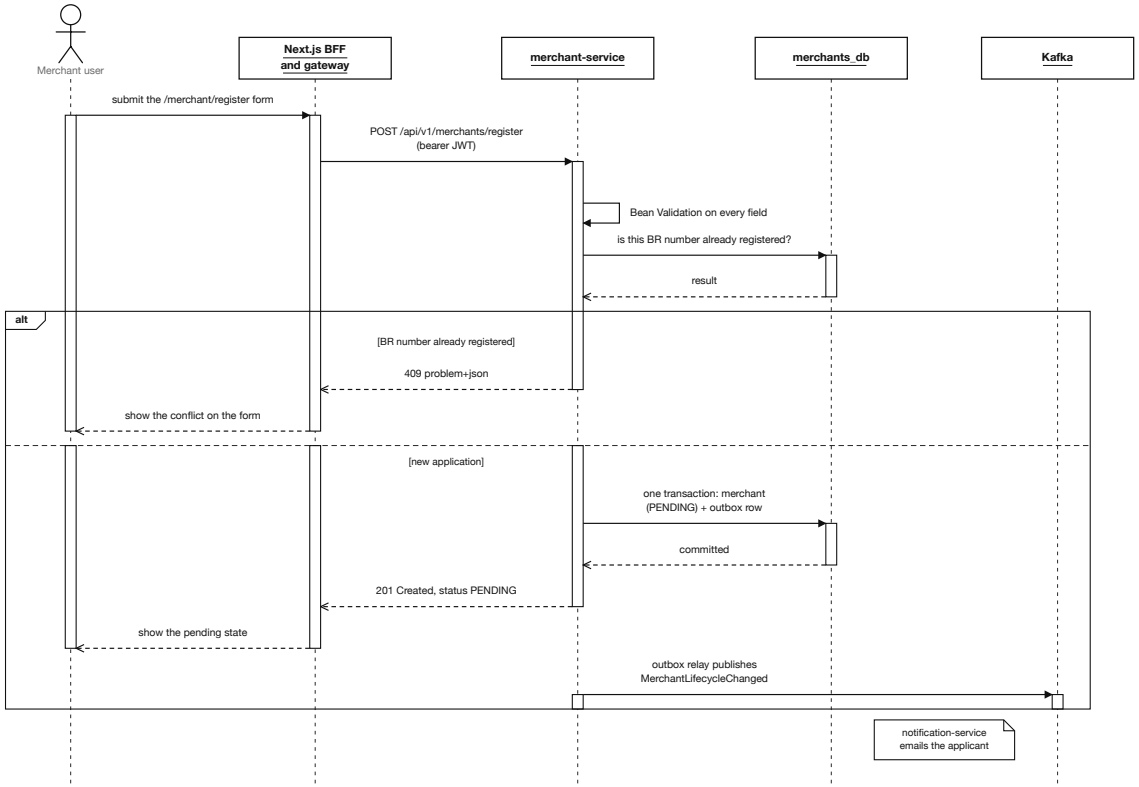
Numbers match the pattern table. Patterns 1 to 5 carry events; patterns 6 to 10 serve requests that need an immediate answer.

#	PATTERN	WHERE USED	WHY
1	Transactional outbox	merchant-service and parcel-service	The row and its event commit together, so no status change or order is lost or invented.
2	Publish/subscribe (broker)	Kafka `swiftdrop.merchant.lifecycle.v1`, `swiftdrop.parcel.created.v1`	One change feeds several consumers without the producer knowing them.
3	Idempotent consumer	`processed_event` table	Kafka delivers at least once; a repeated event must change nothing.
4	Event-carried state transfer	`merchant_status_cache` in parcel-service	The order API can check APPROVED without a synchronous call.
5	Webhook callback	webhook-service to the merchant (M8)	Merchants learn of changes without polling; retries absorb their outages.
6	OAuth 2.0 client credentials	Merchant system to Keycloak	A machine has no user to log in, and Keycloak can rotate or revoke its access centrally.
7	API gateway with JWT	Spring Cloud Gateway	One protected entry point with per-client rate limits; each service validates the token again.
8	Idempotent receiver	`Idempotency-Key` on order creation	A store can safely retry after a timeout without creating a second parcel.
9	Request/reply with circuit breaker	parcel-service to pricing-service	The order needs a fee before it is accepted, and a pricing outage must fail fast.
10	Adapter over request/reply	`KeycloakAdminAdapter`	Provisioning needs an immediate answer, and the application layer stays independent of Keycloak.

5 Key flows

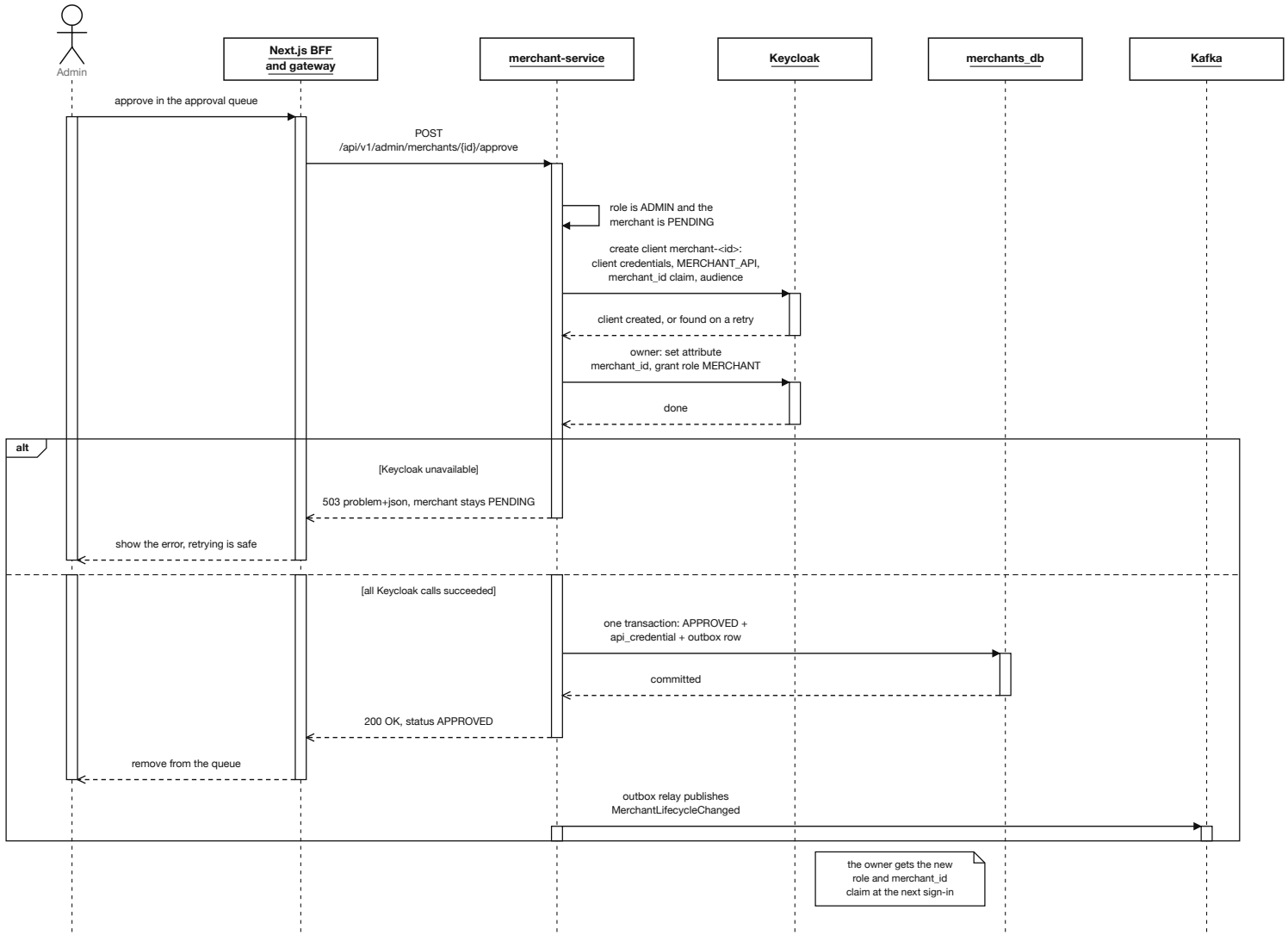
Figures 5 and 6 take a merchant from application to approval. Figure 7 is the required merchant API order flow, and figure 10 shows every decision made on that request. The outbox relay runs after the response, so publishing never delays the caller.

FIGURE 5 **Sequence: a user registers a business**



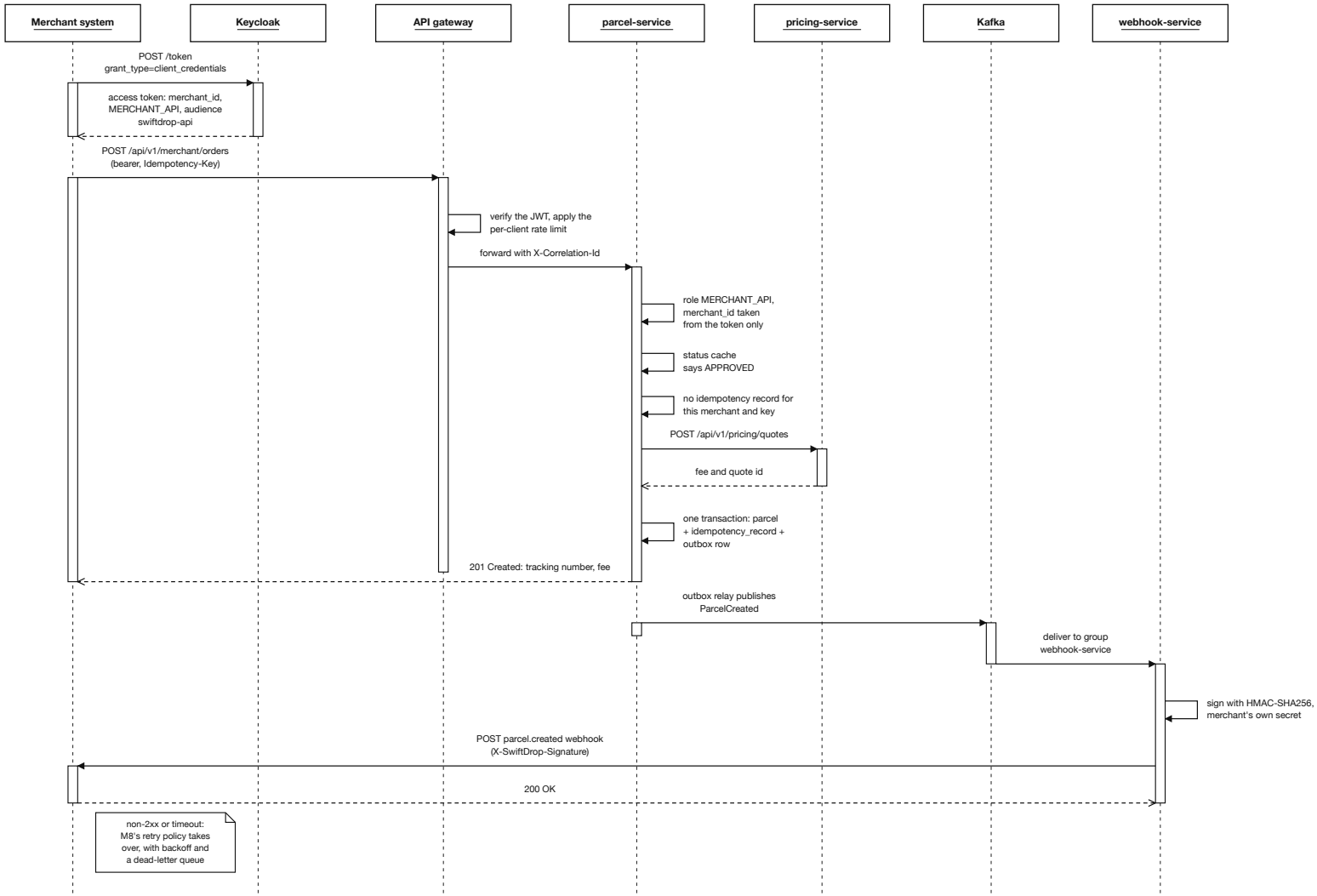
Any signed-in user can apply. The application and its event commit together, and the applicant is told about a duplicate business registration number at once.

FIGURE 6 **Sequence: an admin approves a merchant**



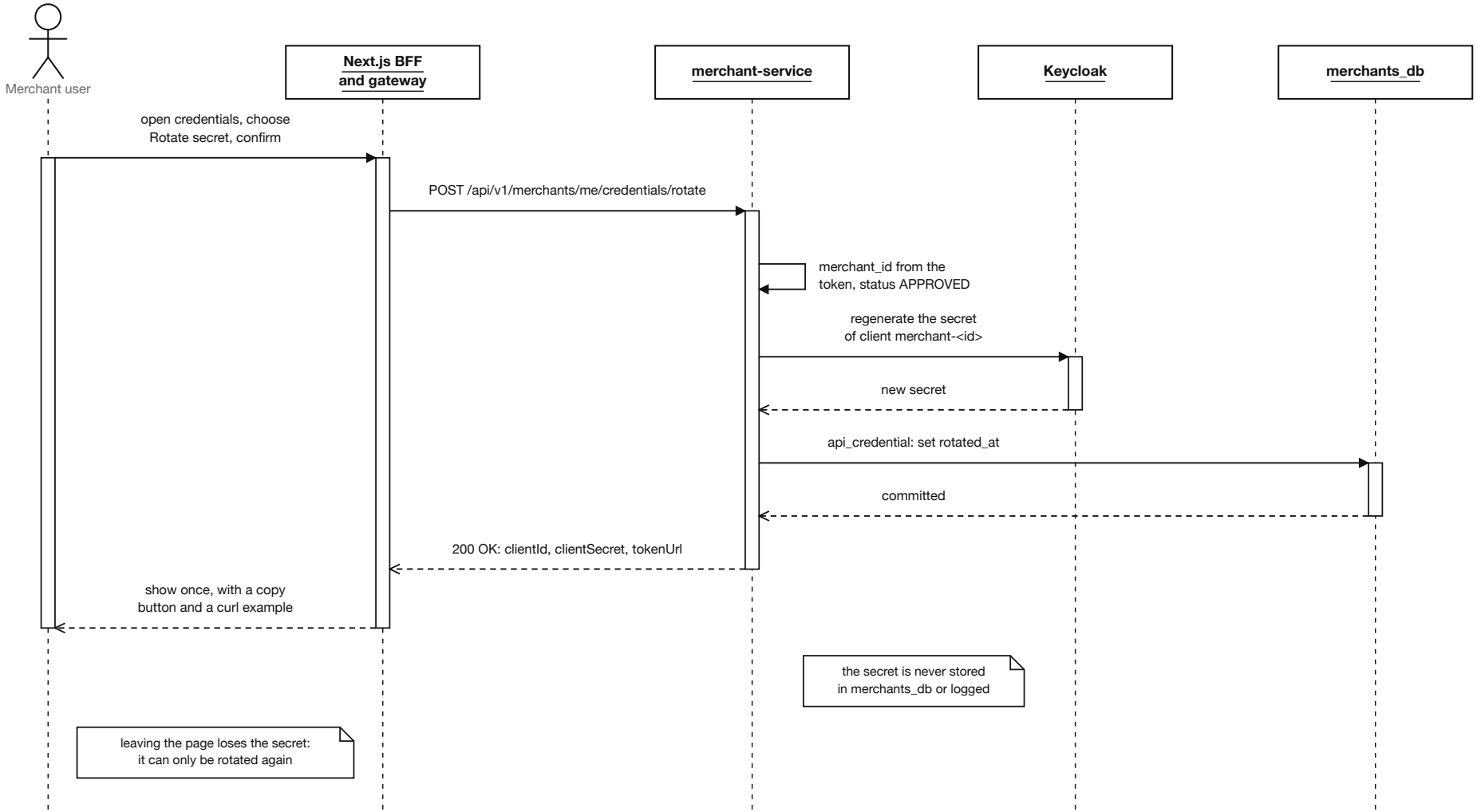
Keycloak is called before the commit and every call is safe to repeat, so a failure leaves the merchant PENDING and the admin simply approves again.

FIGURE 7 **Sequence: a merchant system creates an order through the API**



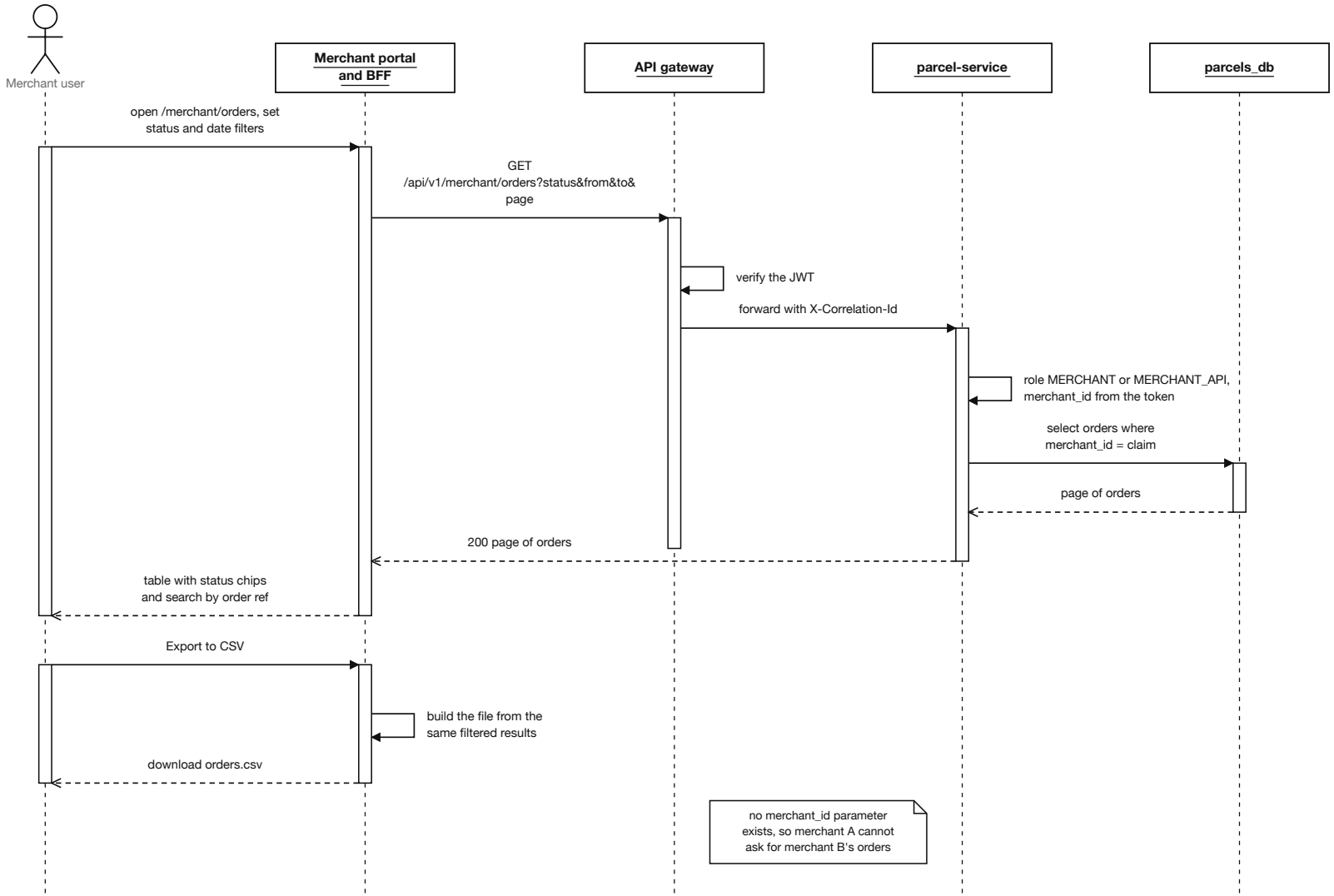
merchant_id always comes from the signed token, never from the request body. The webhook is signed by webhook-service, owned by M8.

FIGURE 8 **Sequence: a merchant rotates the API secret**



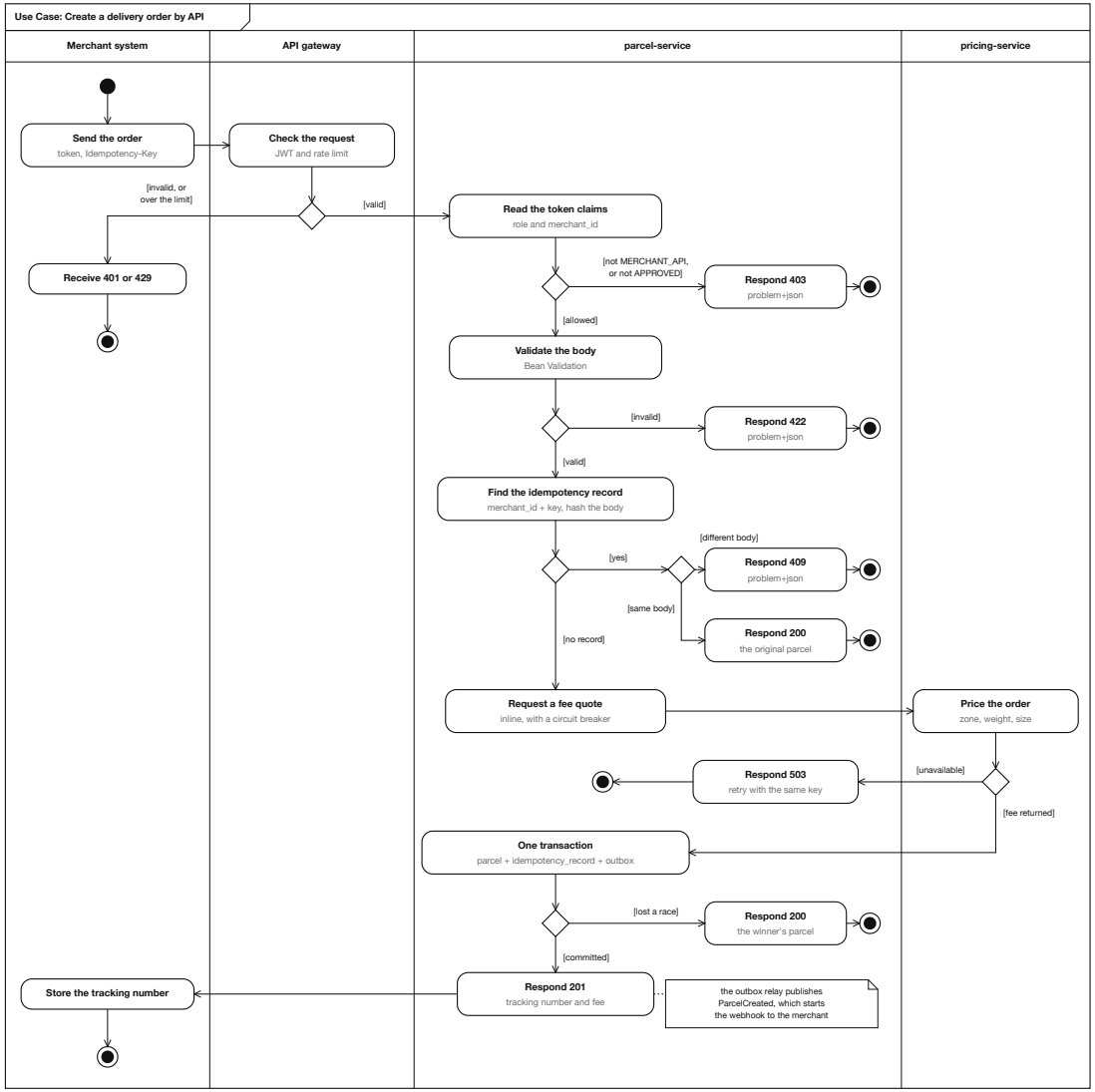
The first secret is issued the same way, after approval, because the approving admin must never see it. Only Keycloak keeps the secret.

FIGURE 9 **Sequence: a merchant views and exports orders**



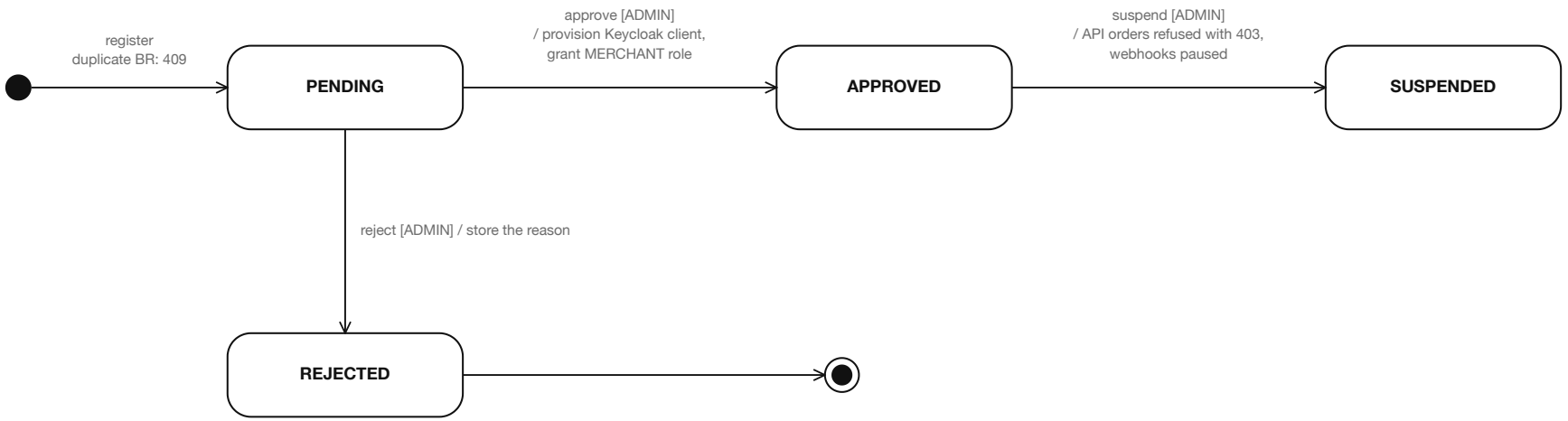
Portal users and API clients read the same endpoint. The merchant_id claim scopes every query, so isolation does not depend on the caller's input.

FIGURE 10 **Activity: how one merchant order request is decided**



Each check that fails ends at its own response. Retrying with the same key can never create a second parcel, even when two requests race.

FIGURE 11 **State diagram: merchant lifecycle**



Only an admin moves a merchant between states, and each change publishes MerchantLifecycleChanged. Reinstating a suspended merchant is outside issue #20.

6 API design note

Versioning

Every route starts with `/api/v1`. Compatible additions, such as a new optional field, stay in v1; a breaking change gets `/api/v2`, and v1 runs alongside it while merchants migrate. Their integrations ship on their own schedule, so a version can never change under them. Events are versioned the same way, in the topic name and the `eventVersion` field.

Idempotency

Order creation requires an `Idempotency-Key` header. The record is keyed on merchant and key, so two merchants can never collide, and it stores a SHA-256 hash of the body. The same key and body return the original parcel with 200; the same key with a different body returns 409. The record commits with the parcel, and its primary key settles a race between two identical requests.

Client credentials

Each approved merchant gets one confidential Keycloak client, `merchant-<id>`. Its token carries the audience `swiftdrop-api`, the role `MERCHANT_API` and a hard-coded `merchant_id` claim, so a client cannot act for another merchant. The secret is shown once and exists only in Keycloak. Rotating replaces it at once, which also ends a leaked secret.

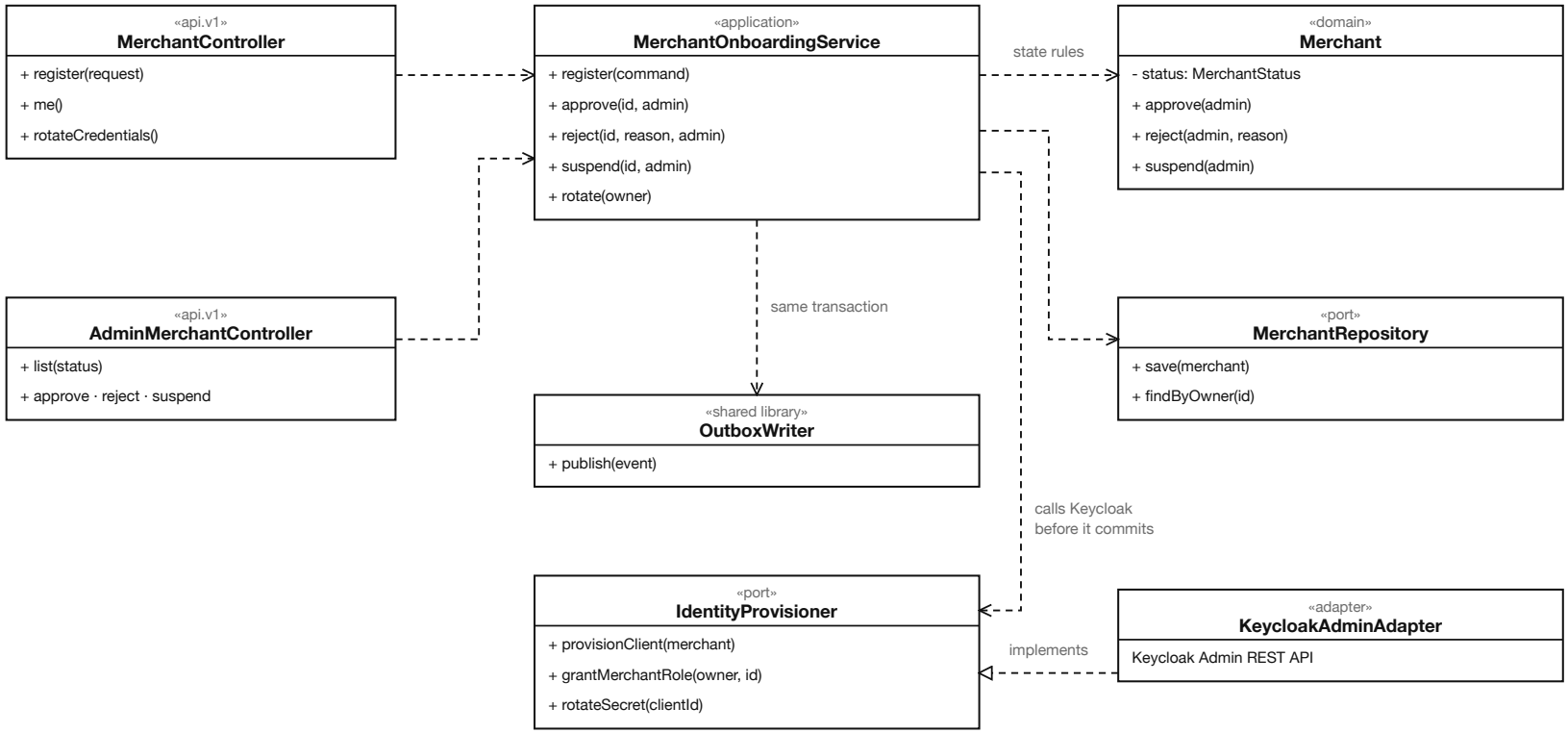
Errors and validation

Errors use RFC 9457 `application/problem+json` with a `traceId`, and never contain stack traces or personal data. Bean Validation checks every body. A row owned by another merchant returns 404, so its existence is not revealed. Phone numbers and addresses are encrypted at rest, and logs are masked.

ENDPOINT	CALLER	MAIN RESPONSES
`POST /api/v1/merchants/register`	Any signed-in user	201 PENDING · 409 duplicate BR number
`GET /api/v1/merchants/me`	Owner	200 profile and status · 404
`POST /api/v1/merchants/me/credentials/rotate`	MERCHANT	200 new secret, shown once
`GET /api/v1/admin/merchants?status=`	ADMIN	200 applications
`POST /api/v1/admin/merchants/{id}/approve · reject · suspend`	ADMIN	200 · 403 non-admin · 422 wrong state
`POST /api/v1/merchant/orders`	MERCHANT_API	201 created · 200 replay · 403 · 409 · 422
`GET /api/v1/merchant/orders`	MERCHANT or MERCHANT_API	200 page of the caller's own orders

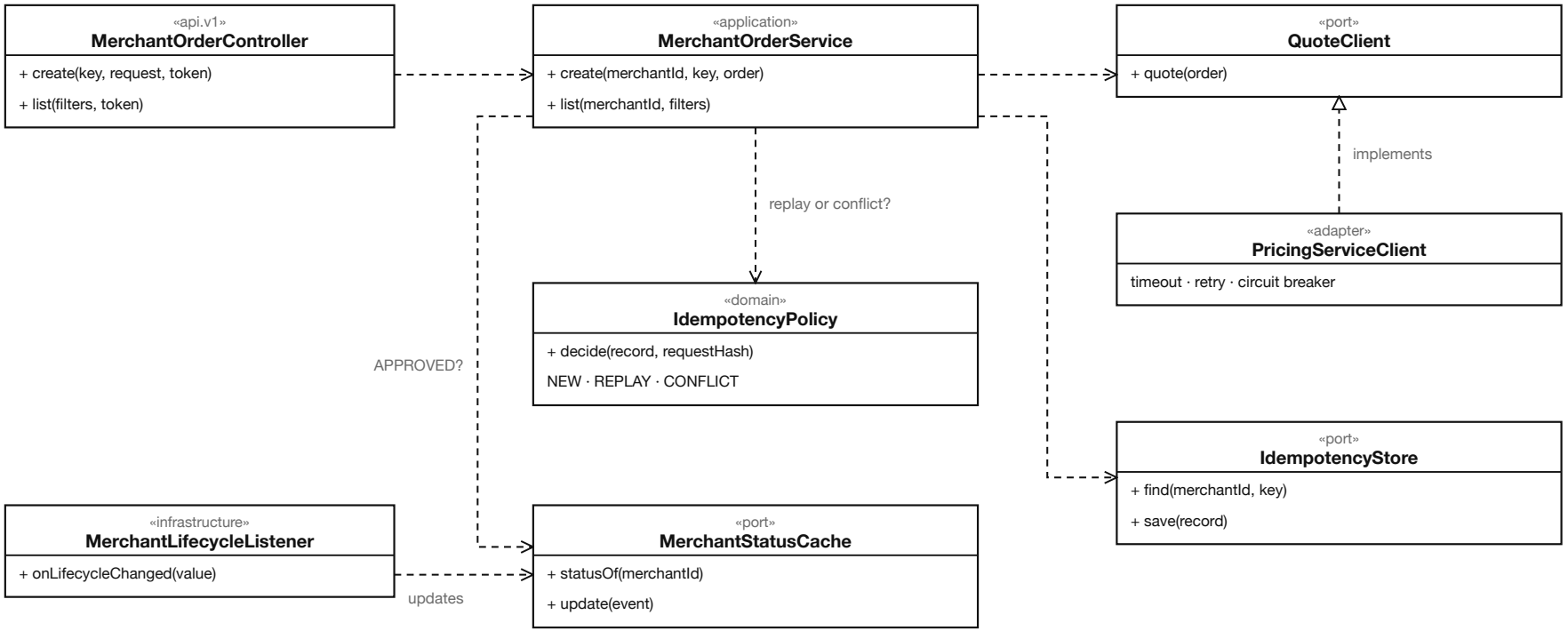
7 Design patterns and principles

FIGURE 12 **Design patterns in merchant-service**



Ports keep the application layer free of Keycloak and JPA; the Merchant class holds the state rules as plain Java. Dashed open arrow: depends on. Dashed hollow triangle: implements.

FIGURE 13 **Design patterns in the merchant order API**



The order service depends on ports for quotes, idempotency records and merchant status; the idempotency decision is a pure domain policy that is simple to unit test.

PATTERN	APPLIED TO
Adapter	`KeycloakAdminAdapter` and `PricingServiceClient` put external systems behind application ports.
State	The `Merchant` class permits only the lifecycle moves in figure 11 and refuses all others.
Repository and ports	`MerchantRepository` and `IdempotencyStore` hide persistence from the application layer.
Strategy-style policy	`IdempotencyPolicy` holds the NEW, REPLAY or CONFLICT rule as one pure decision.
Observer / pub-sub	Notification, webhook and parcel consumers react to `MerchantLifecycleChanged`.
Outbox and idempotent consumer	Shared library `swiftdrop-outbox`: reliable publishing and safe re-delivery.
Circuit breaker	Resilience4j on the pricing call, so an outage returns 503 quickly.
Facade / BFF	Next.js route handlers keep tokens out of the browser for the portal and approval queue.

Principles

- **Single responsibility:** controllers only validate and delegate; state rules live in `Merchant`; Keycloak details live in one adapter.
- **Open/closed:** a new consumer of lifecycle events needs no change to merchant-service.
- **Dependency inversion:** application services depend on ports, and ArchUnit enforces the layers.
- **DRY:** JWT handling, errors, logging and the outbox come from the shared libraries.
- **Least privilege:** machine clients hold only `MERCHANT_API`, and only merchant-service can manage Keycloak clients.
- **Twelve-factor configuration:** Keycloak URLs and client secrets come from environment variables; no secret is stored in code.